skills / jackspace / claudeskillz / statistical-analysis

statistical-analysis

```
$ npx skills add https://github.com/jackspace/claudeskillz --skill statistical-a…
```

SKILL.md

Statistical Analysis

Overview

Statistical analysis is a systematic process for testing hypotheses and quantifying relationships. Conduct hypothesis tests (t-test, ANOVA, chi-square), regression, correlation, and Bayesian analyses with assumption checks and APA reporting. Apply this skill for academic research.

When to Use This Skill

This skill should be used when:

- Conducting statistical hypothesis tests (t-tests, ANOVA, chi-square)
- Performing regression or correlation analyses
- Running Bayesian statistical analyses
- Checking statistical assumptions and diagnostics
- Calculating effect sizes and conducting power analyses
- Reporting statistical results in APA format
- Analyzing experimental or observational data for research

Core Capabilities

1. Test Selection and Planning

- Choose appropriate statistical tests based on research questions and data characteristics
- Conduct a priori power analyses to determine required sample sizes
- Plan analysis strategies including multiple comparison corrections

2. Assumption Checking

- Automatically verify all relevant assumptions before running tests
- Provide diagnostic visualizations (Q-Q plots, residual plots, box plots)
- Recommend remedial actions when assumptions are violated

3. Statistical Testing

- Hypothesis testing: t-tests, ANOVA, chi-square, non-parametric alternatives
- Regression: linear, multiple, logistic, with diagnostics
- Correlations: Pearson, Spearman, with confidence intervals
- Bayesian alternatives: Bayesian t-tests, ANOVA, regression with Bayes Factors

4. Effect Sizes and Interpretation

- Calculate and interpret appropriate effect sizes for all analyses
- Provide confidence intervals for effect estimates
- Distinguish statistical from practical significance

5. Professional Reporting

- Generate APA-style statistical reports
- Create publication-ready figures and tables
- Provide complete interpretation with all required statistics

---

Workflow Decision Tree

Use this decision tree to determine your analysis path:

```
START
│
├── Need to SELECT a statistical test?
│   └── YES → See "Test Selection Guide"
│   └── NO → Continue
│
├── Ready to check ASSUMPTIONS?
│   └── YES → See "Assumption Checking"
│   └── NO → Continue
│
├── Ready to run ANALYSIS?
│   └── YES → See "Running Statistical Tests"
│   └── NO → Continue
│
└── Need to REPORT results?
    └── YES → See "Reporting Results"
```

Test Selection Guide

Quick Reference: Choosing the Right Test

Use `references/test_selection_guide.md` for comprehensive guidance. Quick reference:

Comparing Two Groups:

- Independent, continuous, normal → Independent t-test
- Independent, continuous, non-normal → Mann-Whitney U test
- Paired, continuous, normal → Paired t-test
- Paired, continuous, non-normal → Wilcoxon signed-rank test
- Binary outcome → Chi-square or Fisher's exact test

Comparing 3+ Groups:

- Independent, continuous, normal → One-way ANOVA
- Independent, continuous, non-normal → Kruskal-Wallis test
- Paired, continuous, normal → Repeated measures ANOVA

- Paired, continuous, non-normal → Friedman test

Relationships:

- Two continuous variables → Pearson (normal) or Spearman correlation (non-normal)
- Continuous outcome with predictor(s) → Linear regression
- Binary outcome with predictor(s) → Logistic regression

Bayesian Alternatives: All tests have Bayesian versions that provide:

- Direct probability statements about hypotheses
- Bayes Factors quantifying evidence
- Ability to support null hypothesis
- See `references/bayesian_statistics.md`

Assumption Checking

Systematic Assumption Verification

ALWAYS check assumptions before interpreting test results.

Use the provided `scripts/assumption_checks.py` module for automated checking:

```
from scripts.assumption_checks import comprehensive_assumption_check

# Comprehensive check with visualizations
results = comprehensive_assumption_check(
    data=df,
    value_col='score',
    group_col='group',  # Optional: for group comparisons
    alpha=0.05
)
```

This performs:

1. **Outlier detection** (IQR and z-score methods)
2. **Normality testing** (Shapiro-Wilk test + Q-Q plots)

3. **Homogeneity of variance** (Levene's test + box plots)
4. **Interpretation and recommendations**

Individual Assumption Checks

For targeted checks, use individual functions:

```
from scripts.assumption_checks import (
    check_normality,
    check_normality_per_group,
    check_homogeneity_of_variance,
    check_linearity,
    detect_outliers
)

# Example: Check normality with visualization
result = check_normality(
    data=df['score'],
    name='Test Score',
    alpha=0.05,
    plot=True
)
print(result['interpretation'])
print(result['recommendation'])
```

What to Do When Assumptions Are Violated

Normality violated:

- Mild violation + n > 30 per group → Proceed with parametric test (robust)
- Moderate violation → Use non-parametric alternative
- Severe violation → Transform data or use non-parametric test

Homogeneity of variance violated:

- For t-test → Use Welch's t-test
- For ANOVA → Use Welch's ANOVA or Brown-Forsythe ANOVA
- For regression → Use robust standard errors or weighted least squares

Linearity violated (regression):

- Add polynomial terms

- Transform variables
- Use non-linear models or GAM

See `references/assumptions_and_diagnostics.md` for comprehensive guidance.

Running Statistical Tests

Python Libraries

Primary libraries for statistical analysis:

- **scipy.stats**: Core statistical tests
- **statsmodels**: Advanced regression and diagnostics
- **pingouin**: User-friendly statistical testing with effect sizes
- **pymc**: Bayesian statistical modeling
- **arviz**: Bayesian visualization and diagnostics

Example Analyses

T-Test with Complete Reporting

```
import pingouin as pg
import numpy as np

# Run independent t-test
result = pg.ttest(group_a, group_b, correction='auto')

# Extract results
t_stat = result['T'].values[0]
df = result['dof'].values[0]
p_value = result['p-val'].values[0]
cohens_d = result['cohen-d'].values[0]
ci_lower = result['CI95%'].values[0][0]
ci_upper = result['CI95%'].values[0][1]

# Report
print(f"t({df:.0f}) = {t_stat:.2f}, p = {p_value:.3f}")
print(f"Cohen's d = {cohens_d:.2f}, 95% CI [{ci_lower:.2f}, {ci_upper:.2f}]")
```

ANOVA with Post-Hoc Tests

```
import pingouin as pg

# One-way ANOVA
aov = pg.anova(dv='score', between='group', data=df, detailed=True)
print(aov)

# If significant, conduct post-hoc tests
if aov['p-unc'].values[0] < 0.05:
    posthoc = pg.pairwise_tukey(dv='score', between='group', data=df)
    print(posthoc)

# Effect size
eta_squared = aov['np2'].values[0]  # Partial eta-squared
print(f"Partial η² = {eta_squared:.3f}")
```

Linear Regression with Diagnostics

```
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

# Fit model
X = sm.add_constant(X_predictors)  # Add intercept
model = sm.OLS(y, X).fit()

# Summary
print(model.summary())

# Check multicollinearity (VIF)
vif_data = pd.DataFrame()
vif_data["Variable"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.sh
print(vif_data)

# Check assumptions
residuals = model.resid
fitted = model.fittedvalues

# Residual plots
import matplotlib.pyplot as plt
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Residuals vs fitted
axes[0, 0].scatter(fitted, residuals, alpha=0.6)
axes[0, 0].axhline(y=0, color='r', linestyle='--')
axes[0, 0].set_xlabel('Fitted values')
axes[0, 0].set_ylabel('Residuals')
axes[0, 0].set_title('Residuals vs Fitted')
```

```
# Q-Q plot
from scipy import stats
stats.probplot(residuals, dist="norm", plot=axes[0, 1])
axes[0, 1].set_title('Normal Q-Q')

# Scale-Location
axes[1, 0].scatter(fitted, np.sqrt(np.abs(residuals / residuals.std())), alpha
axes[1, 0].set_xlabel('Fitted values')
axes[1, 0].set_ylabel('√|Standardized residuals|')
axes[1, 0].set_title('Scale-Location')

# Residuals histogram
axes[1, 1].hist(residuals, bins=20, edgecolor='black', alpha=0.7)
axes[1, 1].set_xlabel('Residuals')
axes[1, 1].set_ylabel('Frequency')
axes[1, 1].set_title('Histogram of Residuals')

plt.tight_layout()
plt.show()
```

Bayesian T-Test

```
import pymc as pm
import arviz as az
import numpy as np

with pm.Model() as model:
    # Priors
    mu1 = pm.Normal('mu_group1', mu=0, sigma=10)
    mu2 = pm.Normal('mu_group2', mu=0, sigma=10)
    sigma = pm.HalfNormal('sigma', sigma=10)

    # Likelihood
    y1 = pm.Normal('y1', mu=mu1, sigma=sigma, observed=group_a)
    y2 = pm.Normal('y2', mu=mu2, sigma=sigma, observed=group_b)

    # Derived quantity
    diff = pm.Deterministic('difference', mu1 - mu2)

    # Sample
    trace = pm.sample(2000, tune=1000, return_inferencedata=True)

# Summarize
print(az.summary(trace, var_names=['difference']))

# Probability that group1 > group2
prob_greater = np.mean(trace.posterior['difference'].values > 0)
```

```
print(f"P(μ₁ > μ₂ | data) = {prob_greater:.3f}")

# Plot posterior
az.plot_posterior(trace, var_names=['difference'], ref_val=0)
```

Effect Sizes

Always Calculate Effect Sizes

Effect sizes quantify magnitude, while p-values only indicate existence of an effect.

See `references/effect_sizes_and_power.md` for comprehensive guidance.

Quick Reference: Common Effect Sizes

Test	Effect Size	Small	Medium	Large
T-test	Cohen's d	0.20	0.50	0.80
ANOVA	η^2_p	0.01	0.06	0.14
Correlation	r	0.10	0.30	0.50
Regression	R^2	0.02	0.13	0.26
Chi-square	Cramér's V	0.07	0.21	0.35

Important: Benchmarks are guidelines. Context matters!

Calculating Effect Sizes

Most effect sizes are automatically calculated by pingouin:

```
# T-test returns Cohen's d
result = pg.ttest(x, y)
d = result['cohen-d'].values[0]

# ANOVA returns partial eta-squared
aov = pg.anova(dv='score', between='group', data=df)
eta_p2 = aov['np2'].values[0]

# Correlation: r is already an effect size
```

```
corr = pg.corr(x, y)
r = corr['r'].values[0]
```

Confidence Intervals for Effect Sizes

Always report CIs to show precision:

```
from pingouin import compute_effsize_from_t

# For t-test
d, ci = compute_effsize_from_t(
    t_statistic,
    nx=len(group1),
    ny=len(group2),
    eftype='cohen'
)
print(f"d = {d:.2f}, 95% CI [{ci[0]:.2f}, {ci[1]:.2f}]")
```

Power Analysis

A Priori Power Analysis (Study Planning)

Determine required sample size before data collection:

```
from statsmodels.stats.power import (
    tt_ind_solve_power,
    FTestAnovaPower
)

# T-test: What n is needed to detect d = 0.5?
n_required = tt_ind_solve_power(
    effect_size=0.5,
    alpha=0.05,
    power=0.80,
    ratio=1.0,
    alternative='two-sided'
)
print(f"Required n per group: {n_required:.0f}")

# ANOVA: What n is needed to detect f = 0.25?
anova_power = FTestAnovaPower()
n_per_group = anova_power.solve_power(
```

```
    effect_size=0.25,
    ngroups=3,
    alpha=0.05,
    power=0.80
)
print(f"Required n per group: {n_per_group:.0f}")
```

Sensitivity Analysis (Post-Study)

Determine what effect size you could detect:

```
# With n=50 per group, what effect could we detect?
detectable_d = tt_ind_solve_power(
    effect_size=None,  # Solve for this
    nobs1=50,
    alpha=0.05,
    power=0.80,
    ratio=1.0,
    alternative='two-sided'
)
print(f"Study could detect d ≥ {detectable_d:.2f}")
```

Note: Post-hoc power analysis (calculating power after study) is generally not recommended. Use sensitivity analysis instead.

See `references/effect_sizes_and_power.md` for detailed guidance.

Reporting Results

APA Style Statistical Reporting

Follow guidelines in `references/reporting_standards.md`.

Essential Reporting Elements

1. **Descriptive statistics**: M, SD, n for all groups/variables
2. **Test statistics**: Test name, statistic, df, exact p-value
3. **Effect sizes**: With confidence intervals
4. **Assumption checks**: Which tests were done, results, actions taken

5. **All planned analyses**: Including non-significant findings

Example Report Templates

Independent T-Test

```
Group A (n = 48, M = 75.2, SD = 8.5) scored significantly higher than
Group B (n = 52, M = 68.3, SD = 9.2), t(98) = 3.82, p < .001, d = 0.77,
95% CI [0.36, 1.18], two-tailed. Assumptions of normality (Shapiro-Wilk:
Group A W = 0.97, p = .18; Group B W = 0.96, p = .12) and homogeneity
of variance (Levene's F(1, 98) = 1.23, p = .27) were satisfied.
```

One-Way ANOVA

```
A one-way ANOVA revealed a significant main effect of treatment condition
on test scores, F(2, 147) = 8.45, p < .001, η²_p = .10. Post hoc
comparisons using Tukey's HSD indicated that Condition A (M = 78.2,
SD = 7.3) scored significantly higher than Condition B (M = 71.5,
SD = 8.1, p = .002, d = 0.87) and Condition C (M = 70.1, SD = 7.9,
p < .001, d = 1.07). Conditions B and C did not differ significantly
(p = .52, d = 0.18).
```

Multiple Regression

```
Multiple linear regression was conducted to predict exam scores from
study hours, prior GPA, and attendance. The overall model was significant,
F(3, 146) = 45.2, p < .001, R² = .48, adjusted R² = .47. Study hours
(B = 1.80, SE = 0.31, β = .35, t = 5.78, p < .001, 95% CI [1.18, 2.42])
and prior GPA (B = 8.52, SE = 1.95, β = .28, t = 4.37, p < .001,
95% CI [4.66, 12.38]) were significant predictors, while attendance was
not (B = 0.15, SE = 0.12, β = .08, t = 1.25, p = .21, 95% CI [-0.09, 0.39]).
Multicollinearity was not a concern (all VIF < 1.5).
```

Bayesian Analysis

```
A Bayesian independent samples t-test was conducted using weakly
informative priors (Normal(0, 1) for mean difference). The posterior
distribution indicated that Group A scored higher than Group B
(M_diff = 6.8, 95% credible interval [3.2, 10.4]). The Bayes Factor
BF₁₀ = 45.3 provided very strong evidence for a difference between
groups, with a 99.8% posterior probability that Group A's mean exceeded
Group B's mean. Convergence diagnostics were satisfactory (all R̂ < 1.01,
ESS > 1000).
```

Bayesian Statistics

When to Use Bayesian Methods

Consider Bayesian approaches when:

- You have prior information to incorporate
- You want direct probability statements about hypotheses
- Sample size is small or planning sequential data collection
- You need to quantify evidence for the null hypothesis
- The model is complex (hierarchical, missing data)

See `references/bayesian_statistics.md` for comprehensive guidance on:

- Bayes' theorem and interpretation
- Prior specification (informative, weakly informative, non-informative)
- Bayesian hypothesis testing with Bayes Factors
- Credible intervals vs. confidence intervals
- Bayesian t-tests, ANOVA, regression, and hierarchical models
- Model convergence checking and posterior predictive checks

Key Advantages

1. **Intuitive interpretation**: "Given the data, there is a 95% probability the parameter is in this interval"
2. **Evidence for null**: Can quantify support for no effect
3. **Flexible**: No p-hacking concerns; can analyze data as it arrives
4. **Uncertainty quantification**: Full posterior distribution

---

Resources

This skill includes comprehensive reference materials:

References Directory

- **test_selection_guide.md**: Decision tree for choosing appropriate statistical tests
- **assumptions_and_diagnostics.md**: Detailed guidance on checking and handling assumption violations
- **effect_sizes_and_power.md**: Calculating, interpreting, and reporting effect sizes; conducting power analyses
- **bayesian_statistics.md**: Complete guide to Bayesian analysis methods
- **reporting_standards.md**: APA-style reporting guidelines with examples

Scripts Directory

- **assumption_checks.py**: Automated assumption checking with visualizations
  - `comprehensive_assumption_check()` : Complete workflow
  - `check_normality()` : Normality testing with Q-Q plots
  - `check_homogeneity_of_variance()` : Levene's test with box plots
  - `check_linearity()` : Regression linearity checks
  - `detect_outliers()` : IQR and z-score outlier detection

Best Practices

1. **Pre-register analyses** when possible to distinguish confirmatory from exploratory
2. **Always check assumptions** before interpreting results
3. **Report effect sizes** with confidence intervals
4. **Report all planned analyses** including non-significant results
5. **Distinguish statistical from practical significance**
6. **Visualize data** before and after analysis
7. **Check diagnostics** for regression/ANOVA (residual plots, VIF, etc.)
8. **Conduct sensitivity analyses** to assess robustness
9. **Share data and code** for reproducibility
10. **Be transparent** about violations, transformations, and decisions

Common Pitfalls to Avoid

1. **P-hacking**: Don't test multiple ways until something is significant
2. **HARKing**: Don't present exploratory findings as confirmatory
3. **Ignoring assumptions**: Check them and report violations
4. **Confusing significance with importance**: $p < .05 \neq$ meaningful effect
5. **Not reporting effect sizes**: Essential for interpretation
6. **Cherry-picking results**: Report all planned analyses
7. **Misinterpreting p-values**: They're NOT probability that hypothesis is true
8. **Multiple comparisons**: Correct for family-wise error when appropriate
9. **Ignoring missing data**: Understand mechanism (MCAR, MAR, MNAR)
10. **Overinterpreting non-significant results**: Absence of evidence ≠ evidence of absence

---

Getting Started Checklist

When beginning a statistical analysis:

- [ ] Define research question and hypotheses
- [ ] Determine appropriate statistical test (use test_selection_guide.md)
- [ ] Conduct power analysis to determine sample size
- [ ] Load and inspect data
- [ ] Check for missing data and outliers
- [ ] Verify assumptions using assumption_checks.py
- [ ] Run primary analysis
- [ ] Calculate effect sizes with confidence intervals
- [ ] Conduct post-hoc tests if needed (with corrections)
- [ ] Create visualizations
- [ ] Write results following reporting_standards.md

- [ ] Conduct sensitivity analyses
- [ ] Share data and code

Support and Further Reading

For questions about:

- **Test selection**: See references/test_selection_guide.md
- **Assumptions**: See references/assumptions_and_diagnostics.md
- **Effect sizes**: See references/effect_sizes_and_power.md
- **Bayesian methods**: See references/bayesian_statistics.md
- **Reporting**: See references/reporting_standards.md

Key textbooks:

- Cohen, J. (1988). *Statistical Power Analysis for the Behavioral Sciences*
- Field, A. (2013). *Discovering Statistics Using IBM SPSS Statistics*
- Gelman, A., & Hill, J. (2006). *Data Analysis Using Regression and Multilevel/Hierarchical Models*
- Kruschke, J. K. (2014). *Doing Bayesian Data Analysis*

Online resources:

- APA Style Guide: https://apastyle.apa.org/
- Statistical Consulting: Cross Validated (stats.stackexchange.com)

WEEKLY INSTALLS

14

REPOSITORY

jackspace/claudeskillz

GITHUB STARS

☆ 9

FIRST SEEN

Jan 24, 2026

SECURITY AUDITS

Gen Agent Trust Hub	PASS
Socket	PASS
Snyk	PASS

INSTALLED ON

claude-code	12
opencode	11
codex	10
antigravity	9
gemini-cli	9
cursor	9